



AI with Rav

Learn AI the way you actually think.



DAY 8 of 30

PYTHON

Type Conversion & Errors — translating between box types

WHAT YOU'LL LEARN TODAY

- › Converting = changing a value's type
- › The three converters `int()` `float()` `str()`
- › `int()` drops the decimal
- › Errors are helpful messages
- › The three errors you'll meet



Type Conversion & Errors — translating between box types

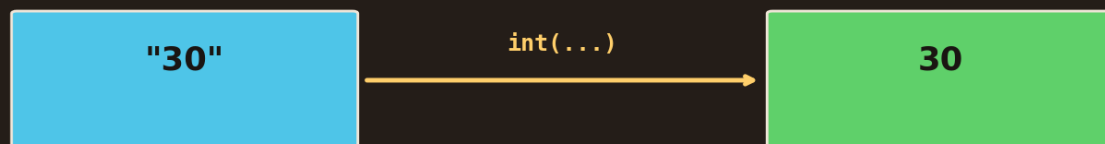
In One Line: Values come in types — text, whole numbers, decimals. Often you need to translate between them (that pesky `input()` gives text!). Today you learn `int()`, `float()`, `str()` to convert safely — and how to read the error message Python shows when a type is wrong. This finishes Part 1.

Start here: types don't always match

You've hit this already — `input()` gives you **text**, but you often need a **number**. Or you have a number and want to glue it into a sentence (which needs text). Values have a **type**, and sometimes the type in your hand isn't the type you need. The fix is **conversion**: translate the value from one type to another.

Converting = changing the type

Converting = translating a value into another type



`int("30")` turns the TEXT `"30"` into the NUMBER `30` you can do maths with.

The value stays the same — only its TYPE changes.

Converting translates a value into another type. The text `"30"` (str) becomes the number `30` (int) via `int(...)`. The value stays the same — only its type changes, so you can now do maths with it.

Converting keeps the **value** but changes its **type** — the box's contents look the same, but now you can *use* them differently:

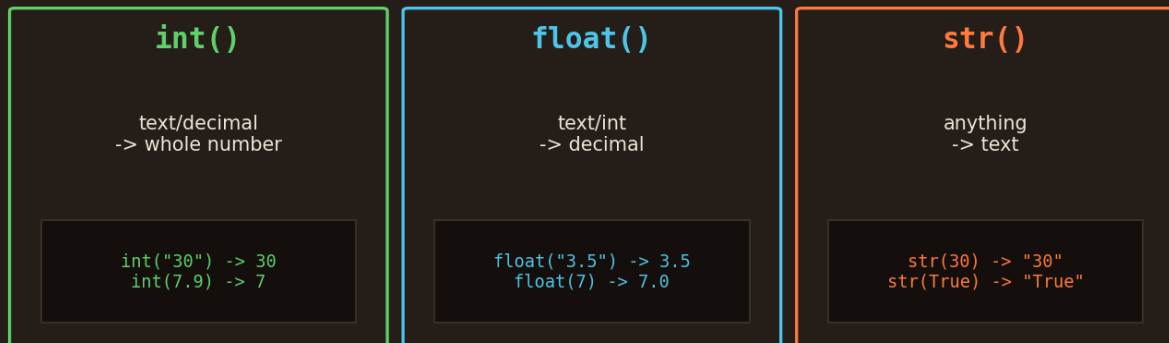
```
text = "30"          # this is text (a string)
number = int(text)   # convert it to a whole number
print(number + 1)    # -> 31 (now maths works!)
```



`int("30")` reads the text `"30"` and hands back the actual number `30`. The value didn't change — but its *type* did, so now `+ 1` works. This is exactly why we wrote `int(input(...))` on Day 5: input gives text, we convert it to a number.

The three everyday converters

The three everyday converters



int() drops the decimal (does NOT round): int(7.9) is 7, not 8.

Three converters: `int()` turns text/decimal into a whole number (`int("30")->30`, `int(7.9)->7`), `float()` turns text/int into a decimal (`float("3.5")->3.5`, `float(7)->7.0`), `str()` turns anything into text (`str(30)->"30"`). `int()` drops the decimal, it does not round.

Three functions do almost all the converting you'll ever need — each named after the type it makes:

- `int(x)` → make a **whole number**: `int("30")` → `30`.
- `float(x)` → make a **decimal**: `float("3.5")` → `3.5`.
- `str(x)` → make **text**: `str(30)` → `"30"`.

<code>str(30)</code>	# -> <code>"30"</code>	number to text (for messages)
<code>float("3.5")</code>	# -> <code>3.5</code>	text to decimal
<code>int("100")</code>	# -> <code>100</code>	text to whole number

Notice the pattern: the function's name **is** the type you want. Need text? `str()`. Need a whole number? `int()`. This is how you fix the "can't add text and number" problem from Day 3 — convert first, then combine.

Watch out: int() drops the decimal



Three errors you WILL meet (and their fix)

TypeError

```
"age " + 30
```

mixing text and number
fix: `str(30)`, or use commas / f-string

ValueError

```
int("hello")
```

converting text that isn't a number
fix: check the text first

NameError

```
print(scoree)
```

a typo in a variable name
fix: fix the spelling: `score`

Three common errors and fixes: `TypeError` from `"age " + 30` (mixing text and number, fix with `str(30)`), `ValueError` from `int("hello")` (converting text that isn't a number), `NameError` from `print(scoree)` (a typo in a variable name, fix the spelling).

One surprise worth calling out before we hit errors:

```
print(int(7.9))    # -> 7    (NOT 8!)
```

`int()` **chops off** the decimal part — it does **not** round. `int(7.9)` is `7`, not `8`. If you actually want rounding, use `round(7.9)` → `8`. Also: `int("hello")` will **error** — you can only convert text that *looks* like a number. `int("30")` works; `int("thirty")` does not.

Errors are helpful messages, not disasters

An error is a helpful message, not a disaster

```
>>> "age " + 30
```

the TYPE of problem

```
TypeError: can only concatenate  
str (not "int") to str
```

what actually went wrong

It tells you: you tried to glue text and a number. Fix: `str(30)` first.

Read the **LAST** line first — it names the problem. Errors are clues, not scolding.



An error is a helpful message. `"age " + 30` gives a `TypeError: can only concatenate str to str`. The error names the TYPE of problem and what went wrong. Read the last line first — it names the problem. Fix: `str(30)`.

Every coder — even experts — sees errors constantly. An error isn't Python scolding you; it's Python **telling you exactly what went wrong** so you can fix it:

```
print("age " + 30)
# TypeError: can only concatenate str (not "int") to str
```

Don't panic at the red text — **read it**. The **last line** names the problem: ``TypeError`` (a type mismatch) and the detail "can only add str to str" — meaning you tried to glue text and a number. The fix is right there: convert the number first with ``str(30)``, or just use commas in ``print``. Errors are clues, not failure.

The three errors you'll meet

Ninety percent of your early errors are one of these three — learn to recognise them and you'll fix them in seconds:

- **TypeError** → wrong *types* together, like `"age " + 30``. Fix: convert (``str(30)``) or use commas.
- **ValueError** → converting text that isn't a valid number, like ``int("hello")``. Fix: check the text.
- **NameError** → a **typo** in a variable name, like ``print(scoree)``. Fix: correct the spelling.

Make a habit: when you see red, **read the last line, spot which of these it is, and the fix usually follows**. Errors feel scary on Day 8 and boring by Day 30 — because you'll have read hundreds and know they're just Python pointing at the exact line to fix. That calm is what separates a beginner from an engineer.

Part 1 complete — the foundation is laid



Part 1 complete — the foundation is laid



You can now store data, do maths, talk to the user, and make decisions.

Next (Part 2): Collections — lists & dictionaries — and Loops.

Part 1 complete: Days 1-8 all done — Why Python, Variables, Strings, Numbers, Input/Output, Booleans, If/Elif/Else, Convert & Errors. You can now store data, do maths, talk to the user, and make decisions. Next in Part 2: Collections (lists and dictionaries) and Loops.

Take a breath — you just finished the **entire foundation** of Python. In eight days you learned to store data, work with text and numbers, talk to the user, make decisions, and handle errors. That's genuinely enough to write small, useful programs. Everything from here builds on these blocks.

Recap — the 20-second version

- **Converting** changes a value's **type** (not its value): text ↔ number.
- ``int()`` → whole number, ``float()`` → decimal, ``str()`` → text.
- ``int()`` **drops** the decimal (doesn't round); only converts text that looks numeric.
- An **error** is Python telling you what's wrong — read the **last line** first.
- The big three: **TypeError** (wrong types), **ValueError** (bad conversion), **NameError** (typo).

Next up — Part 2 begins with Video 9: Lists. So far each variable held ONE value. But real data comes in groups — a class of students, a shopping cart, a month of temperatures. Next we learn Lists: one box that holds many values in order. This is where Python gets powerful. See you tomorrow.

